Northwind Order Management Database

Functional Requirements and Behavior Specification

Version: 1.0

Target system: PostgreSQL 16 with the Northwind schema (northwind_psql)

Scope: Expected functionality, supported SQL behaviors, boundary conditions, configuration options and user visible semantics

Status: Every query result and error code in this document was executed and confirmed on PostgreSQL 16.15

1. Introduction

1.1 Purpose

This document is the central reference for the Northwind order management database. It states what the database stores, how it is expected to behave when queried and modified, which rules it enforces, where its limits are, and what each value means to a person reading the data. It is written so that test scenarios can be derived directly from its statements.

1.2 Scope

The specification covers the 14 tables of the Northwind schema, the integrity rules declared on them, the results of standard queries over the shipped sample data, the error behavior of invalid operations, the deployment settings in the provided Docker configuration, and the meaning of special values such as NULL dates and status flags. Application screens, reporting tools and performance targets are out of scope.

1.3 Conventions

- Each requirement has an identifier such as `REQ-ORD-03`. The prefix names the area of the system.
- Requirements marked [Enforced] are either guaranteed by the database through a key, a NOT NULL rule, a data type or a foreign key, in which case a violation is rejected with an error, or are fixed facts of the shipped sample data that a query confirms.
- Requirements marked [Expected] are business rules that the data follows but the database does not check. A violating statement is accepted, so any process that writes data must enforce these rules itself.
- Error outcomes are given as PostgreSQL SQLSTATE codes, for example `23505` for a duplicate key.
- Example result values are shown exactly as PostgreSQL returns them for the shipped sample data.

2. System Overview

Northwind models a small food import and export company. Customers place orders, each order is taken by an employee, contains one or more product lines and is delivered by a shipper. Products belong to categories and are supplied by suppliers. Employees are assigned to sales territories, which are grouped into regions.

REQ-SYS-01 The database shall run on PostgreSQL version 16 and shall be created entirely from the single script `northwind.sql`. *[Expected]*

REQ-SYS-02 After the script is loaded, the database shall contain exactly the 14 tables and row counts listed in Table 1. *[Enforced]*

Table	Rows	Content
categories	8	Product groups such as Beverages and Seafood
products	77	Items for sale with price, stock and status
suppliers	29	Companies that supply products
customers	91	Companies that place orders

Table	Rows	Content
orders	830	Order headers with dates, shipper and delivery address
order_details	2155	Order lines: one product per line
employees	9	Sales staff with a reporting hierarchy
shippers	6	Delivery companies
region	4	Eastern, Western, Northern, Southern
territories	53	Sales territories, each in one region
employee_territories	49	Assignment of employees to territories
us_states	51	Reference list of US states
customer_demographics	0	Customer type definitions (empty)
customer_customer_demo	0	Customer to type mapping (empty)

Table 1. Tables and row counts of the shipped sample data.

REQ-SYS-03 The sample orders shall cover order dates from 1996-07-04 to 1998-05-06, with 152 orders in 1996, 408 in 1997 and 270 in 1998. *[Enforced]*

3. Expected Functionality by Entity

3.1 Categories and Products

REQ-PRD-01 Every product shall have a unique `product_id` and a `product_name` that is not NULL and is at most 40 characters long. An empty name is not NULL, so the database accepts it. *[Enforced]*

REQ-PRD-02 When `category_id` or `supplier_id` is set, it shall reference an existing category or supplier. Inserting a product with `category_id` 99 shall fail with SQLSTATE `23503`. Both columns allow NULL, so a product with no category is accepted, although every sample product has both. *[Enforced]*

REQ-PRD-03 The `discontinued` column shall hold 1 for a product that is no longer sold and 0 for an active product. The sample data contains 67 active and 10 discontinued products. *[Expected]*

REQ-PRD-04 Unit prices shall be positive. In the sample data the cheapest product costs 2.5 and the most expensive, Côte de Blaye, costs 263.5. The average product price is 28.8339. *[Expected]*

REQ-PRD-05 A product whose `units_in_stock` is below its `reorder_level` needs restocking. The sample data contains 18 such products, and 5 products have zero units in stock. *[Expected]*

REQ-PRD-06 The number of products per category shall be: Beverages 12, Condiments 12, Confections 13, Dairy Products 10, Grains/Cereals 7, Meat/Poultry 6, Produce 5, Seafood 12. *[Enforced]*

REQ-PRD-07 Category names shall be at most 15 characters long. Inserting the category name `Fresh Bakery Goods` (18 characters) shall fail with SQLSTATE `22001`. Names are unique in the sample data, but the database does not require uniqueness. *[Enforced]*

3.2 Suppliers

REQ-SUP-01 Every supplier shall have a unique `supplier_id` and a `company_name` of at most 40 characters. *[Enforced]*

REQ-SUP-02 A supplier that is referenced by at least one product shall not be deletable while that reference exists. *[Enforced]*

3.3 Customers

REQ-CUS-01 Every customer shall be identified by a `customer_id` of at most 5 characters, for example `ALFKI`. An identifier of 6 characters such as `ABCDEF` shall be rejected with SQLSTATE `22001`. *[Enforced]*

REQ-CUS-02 Customer identifiers shall be compared case sensitively. Searching orders for `ALFKI` shall return 6 orders, while searching for `alfki` shall return 0. *[Enforced]*

REQ-CUS-03 Customer identifiers are conventionally written in upper case. The database does not check this, so inserting a customer with identifier `alfki` is accepted as a separate customer. *[Expected]*

REQ-CUS-04 A customer with at least one order shall not be deletable. Deleting customer `ALFKI` shall fail with SQLSTATE `23503`. A customer with no orders, such as `FISSA`, shall be deletable. *[Enforced]*

REQ-CUS-05 Exactly two customers, `FISSA` and `PARIS`, shall have no orders. The remaining 89 customers each have at least one order. *[Enforced]*

REQ-CUS-06 Customers are located in 21 countries. The countries with the most customers are USA with 13, Germany with 11 and France with 11. *[Enforced]*

3.4 Employees and Territories

REQ-EMP-01 Every employee shall have a unique `employee_id`, a `last_name` of at most 20 characters and a `first_name` of at most 10 characters. *[Enforced]*

REQ-EMP-02 The `reports_to` column shall reference the manager of the employee. A value that does not match an existing employee, such as 99, shall be rejected with SQLSTATE `23503`. *[Enforced]*

REQ-EMP-03 Exactly one employee, Andrew Fuller (Vice President, Sales), shall have no manager, shown by a NULL `reports_to`. Employees 1, 3, 4, 5 and 8 report to employee 2, and employees 6, 7 and 9 report to employee 5. *[Enforced]*

REQ-EMP-04 An employee who manages other employees or who has taken orders shall not be deletable. Deleting employee 2 shall fail with SQLSTATE `23503`. *[Enforced]*

REQ-EMP-05 Each territory shall belong to exactly one region. Region Eastern has 19 territories, Western 15, Northern 11 and Southern 8. Deleting a region that has territories shall fail with SQLSTATE `23503`. *[Enforced]*

REQ-EMP-06 Every one of the 9 employees shall be assigned to at least one territory, giving 49 assignments in total. *[Expected]*

3.5 Orders

REQ-ORD-01 Every order shall have a unique `order_id`. Sample order identifiers run from 10248 to 11077 without gaps. *[Enforced]*

REQ-ORD-02 When set, `customer_id`, `employee_id` and `ship_via` shall reference an existing customer, employee and shipper; an unknown value is rejected with SQLSTATE `23503`. These columns allow NULL, so an order with no customer is accepted, although every sample order has all three. *[Enforced]*

REQ-ORD-03 An order whose `shipped_date` is NULL has not yet shipped. The sample data contains 21 unshipped orders, so counting `shipped_date` returns 809 while counting all orders returns 830. *[Expected]*

REQ-ORD-04 An order is late when its `shipped_date` is after its `required_date`. The sample data contains 37 late orders. *[Expected]*

REQ-ORD-05 The shipped date of an order shall not be earlier than its order date. The database does not check this; setting the shipped date of order 10248 to 1990-01-01 is accepted. *[Expected]*

REQ-ORD-06 Every order shall contain at least one order line. The database does not check this; an order header with no lines is accepted. *[Expected]*

REQ-ORD-07 Freight cost shall be positive. Sample freight values range from 0.02 to 1007.64. *[Expected]*

3.6 Order Details

REQ-ODT-01 An order line shall be identified by the pair (`order_id`, `product_id`). Adding product 11 to order 10248 a second time shall fail with SQLSTATE `23505`. *[Enforced]*

REQ-ODT-02 Every order line shall reference an existing order and an existing product. *[Enforced]*

REQ-ODT-03 `unit_price`, `quantity` and `discount` shall all be present. Inserting a line without a discount shall fail with SQLSTATE `23502`. *[Enforced]*

REQ-ODT-04 The discount shall be a fraction between 0 and 1. Sample discounts take only the values 0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.1, 0.15, 0.2 and 0.25. The database does not check the range, so a discount of 1.5 is accepted. *[Expected]*

REQ-ODT-05 Quantity shall be a positive whole number. Sample quantities range from 1 to 130. The database does not check the sign, so a quantity of -5 is accepted. *[Expected]*

REQ-ODT-06 The unit price on an order line records the price at the time of sale and may differ from the current price in `products`. *[Expected]*

REQ-ODT-07 An order shall have between 1 and 25 lines in the sample data. 137 orders have exactly one line. *[Enforced]*

3.7 Shippers and Reference Data

REQ-SHP-01 The database shall contain 6 shippers. Orders use only shippers 1 (Speedy Express, 249 orders), 2 (United Package, 326 orders) and 3 (Federal Shipping, 255 orders). Shippers 4, 5 and 6 have no orders. *[Enforced]*

REQ-REF-01 The tables `customer_demographics` and `customer_customer_demo` shall exist with their keys but contain no rows. *[Enforced]*

REQ-REF-02 The table `us_states` is reference data with 51 rows and has no foreign key link to any other table. *[Enforced]*

4. Supported SQL Behaviors

This section states the results that standard queries shall return on the sample data. These are the behaviors a client application relies on.

4.1 Order Totals

REQ-SQL-01 The value of an order line shall be computed as `unit_price * quantity * (1 - discount)`. The value of an order is the sum of its lines. *[Expected]*

REQ-SQL-02 Order 10248 shall have 3 lines (products 11, 42 and 72) and a total value of 440.00. *[Enforced]*

REQ-SQL-03 Order 10250 shall have a total value of 1552.60. Its line for product 51 (price 42.4, quantity 35, discount 0.15) is worth 1261.40. *[Enforced]*

REQ-SQL-04 The total value of all order lines shall be 1265793.04 when rounded to 2 decimal places. *[Enforced]*

```
SELECT round(sum(unit_price::numeric * quantity
            * (1 - discount::numeric)), 2)
FROM order_details WHERE order_id = 10248;   -- 440.00
```

REQ-SQL-05 The three customers with the highest total order value shall be QUICK (110277.31), ERNSH (104874.98) and SAVEA (104361.95). *[Enforced]*

4.2 Joins

REQ-SQL-06 An inner join from `orders` to `customers` shall return all 830 orders, because no order has a NULL customer. *[Enforced]*

REQ-SQL-07 A left join from `customers` to `orders` that keeps rows with no matching order shall return exactly `FISSA` and `PARIS`. *[Enforced]*

REQ-SQL-08 A self join of `employees` on `reports_to` shall return Andrew as the manager of Nancy (employee 1). *[Enforced]*

REQ-SQL-09 An anti join (NOT EXISTS) of `shippers` against `orders` shall return shippers 4, 5 and 6. *[Enforced]*

4.3 Filtering and Aggregation

REQ-SQL-10 A date range filter on `order_date` between 1997-01-01 and 1997-12-31 shall return 408 orders. Both ends of BETWEEN are included. *[Enforced]*

REQ-SQL-11 Employee 1 shall have taken 55 orders with an order date in 1997. *[Enforced]*

REQ-SQL-12 Filtering products with unit price BETWEEN 10 AND 20 shall return 29 products. *[Enforced]*

REQ-SQL-13 Grouping orders by `ship_country` shall place USA and Germany first, with 122 orders each. *[Enforced]*

REQ-SQL-14 Counting active Beverages products (category 1, discontinued 0) shall return 9. *[Enforced]*

4.4 NULL Semantics

REQ-SQL-15 A comparison with NULL using = shall never be true. `WHERE ship_region = NULL` shall return 0 orders, while `WHERE ship_region IS NULL` shall return 507. *[Enforced]*

REQ-SQL-16 `count(column)` shall ignore NULL values. `count(ship_region)` over orders shall return 323, and 323 plus 507 equals the 830 orders. *[Enforced]*

REQ-SQL-17 Aggregates such as `sum` and `avg` shall skip NULL values rather than treat them as zero. *[Enforced]*

4.5 Text Matching

REQ-SQL-18 Equality on text shall be case sensitive: `'Chai' = 'chai'` is false. *[Enforced]*

REQ-SQL-19 `LIKE` shall be case sensitive and `ILIKE` shall ignore case. On product names, `LIKE 'C%'` returns 9 rows, `LIKE 'c%'` returns 0, and `ILIKE 'c%'` returns 9, because all names start with a capital letter. *[Enforced]*

REQ-SQL-20 Text values shall be stored and returned in UTF8, so names with accents such as `Côte de Blaye` and `Thüringer Rostbratwurst` keep their characters. *[Enforced]*

4.6 Numeric Behavior

REQ-SQL-21 Prices, discounts and freight are stored as `real` (single precision floating point). Results of arithmetic on these columns are approximate: `0.1::real * 3` returns 0.30000000447034836 and is not equal to `0.3::real`. *[Enforced]*

REQ-SQL-22 Money values shall be cast to `numeric` and rounded to 2 decimal places before they are compared or reported. *[Expected]*

REQ-SQL-23 Division of two integers shall truncate: `10/4` returns 2, while `10/4.0` returns 2.5. *[Enforced]*

4.7 Transactions

REQ-SQL-24 Every change made inside a transaction that ends with ROLLBACK shall be undone, leaving all row counts as in Table 1. *[Enforced]*

REQ-SQL-25 A statement that fails inside a transaction shall abort the transaction; further statements are rejected with SQLSTATE `25P02` until ROLLBACK. *[Enforced]*

5. Integrity Rules and Error Behavior

Table 2 lists the rules the database checks and the error each violation produces. Every case was executed inside a transaction that was then rolled back.

Operation	Rule violated	Result
Insert category with existing id 1	Primary key	Error `23505`
Insert category without an id	NOT NULL on key	Error `23502`
Insert product with category 99	Foreign key to categories	Error `23503`
Delete customer ALFKI	Referenced by orders	Error `23503`
Delete customer FISSA	None (no orders)	Accepted
Change order_id of order 10248	Referenced by order_details	Error `23503`
Insert employee reporting to 99	Self foreign key	Error `23503`
Delete employee 2	Referenced by employees and orders	Error `23503`
Delete region 1	Referenced by territories	Error `23503`
Repeat product 11 in order 10248	Composite primary key	Error `23505`
Order line without discount	NOT NULL	Error `23502`
Customer id `ABCDEF`	varchar(5) length	Error `22001`
Quantity 40000	smallint range	Error `22003`

Operation	Rule violated	Result
Order id 32768	smallint range	Error `22003`

Table 2. Enforced rules and their error codes.

REQ-INT-01 All foreign keys shall use the default NO ACTION behavior. Deleting or changing a referenced row shall be rejected; no delete or update shall cascade to child rows. *[Enforced]*

REQ-INT-02 No table defines a default value or an identity column. Every insert shall supply the key value explicitly. *[Enforced]*

REQ-INT-03 No CHECK constraints are defined. All value range rules in this document marked [Expected] rely on the writing application. *[Enforced]*

6. Boundary Conditions

6.1 Numeric Limits

REQ-BND-01 Columns of type `smallint` (all integer identifiers except text keys, plus `quantity`, `units_in_stock`, `units_on_order` and `reorder_level`) shall accept values from -32768 to 32767. The values -32768 and 32767 are accepted, while -32769 and 32768 are rejected with SQLSTATE `22003`. *[Enforced]*

REQ-BND-02 Arithmetic on two smallint values that exceeds 32767, such as `32767::smallint + 1::smallint`, shall fail with "smallint out of range". *[Enforced]*

REQ-BND-03 The highest current order id is 11077, so 21690 more orders can be numbered before the smallint limit of 32767 is reached. *[Expected]*

6.2 Text Length Limits

Column	Max length	Example at the limit
customers.customer_id	5	`ALFKI`
categories.category_name	15	`Dairy Products` (14)
employees.first_name	10	Values up to 10 characters
employees.extension	4	Phone extension digits
products.product_name	40	Values up to 40 characters
customers.country	15	Country names up to 15 characters
us_states.state_abbr	2	TX

REQ-BND-04 On INSERT or UPDATE, a text value one character longer than its column limit shall be rejected with SQLSTATE `22001` and shall not be silently cut. An explicit cast behaves differently: `'ABCDEF'::varchar(5)` returns ABCDE without an error. *[Enforced]*

REQ-BND-05 A text value exactly at the column limit shall be accepted and returned unchanged. *[Enforced]*

6.3 Date Limits

REQ-BND-06 Date columns shall reject impossible calendar dates. The value 1997-02-29 is rejected with "date/time field value out of range" because 1997 is not a leap year. *[Enforced]*

REQ-BND-07 No sample order has an order date after 1998-05-06. A query for later orders shall return 0 rows. *[Enforced]*

6.4 Business Value Limits (not enforced)

REQ-BND-08 Discount shall be at least 0 and less than 1. The database accepts 1.5, which would produce a negative line value. *[Expected]*

REQ-BND-09 Quantity shall be at least 1. The database accepts 0 and negative values. *[Expected]*

REQ-BND-10 `discontinued` shall be 0 or 1. Because the column is an integer, the value 5 is accepted. *[Expected]*

REQ-BND-11 Stock counts shall not be negative. 17 products currently have units on order greater than zero. *[Expected]*

7. Configuration Options

The repository provides a Docker Compose file that starts the database and an administration tool. The settings below are the supported configuration.

Setting	Value	Meaning
Database image	`postgres:latest`	Official PostgreSQL server image
POSTGRES_DB	`northwind`	Name of the created database
POSTGRES_USER	`postgres`	Superuser account
POSTGRES_PASSWORD	`postgres`	Password of that account
Host port	`55432`	Mapped to container port 5432
Init script	`northwind.sql`	Run once when the data volume is empty
pgAdmin port	`5050`	Web administration tool
pgAdmin login	`pgadmin4@pgadmin.org`	Password `postgres`
Data volume	`postgresql_data`	Keeps data across restarts

REQ-CFG-01 Clients outside Docker shall connect on host port 55432 to database `northwind` as user `postgres`. *[Expected]*

REQ-CFG-02 The init script shall run only when the data volume is empty. Restarting the container shall keep existing data; to reload the sample data the volume must be removed. *[Expected]*

REQ-CFG-03 The default session settings shall be: transaction isolation `read committed`, date style `ISO, MDY`, and server encoding `UTF8`. Dates are therefore returned as YYYY-MM-DD. *[Enforced]*

REQ-CFG-04 The default credentials are for local development only and shall be changed before any shared or networked deployment. *[Expected]*

REQ-CFG-05 Because `postgres:latest` changes over time, a deployment that needs repeatable results shall pin the image to a fixed major version such as 16. *[Expected]*

8. User Visible Semantics

This section explains what stored values mean to a person reading reports or screens built on the database.

REQ-SEM-01 Product status shall be displayed as "Discontinued" when `discontinued` is 1 and "Active" when it is 0. Discontinued products remain visible in past orders. *[Expected]*

REQ-SEM-02 An order with a NULL `shipped_date` shall be displayed as "Not shipped". It shall not be displayed as shipped on an empty or default date. *[Expected]*

REQ-SEM-03 An order shipped after its required date shall be displayed as "Late". *[Expected]*

REQ-SEM-04 A NULL `ship_region`, `region` or `fax` means the value does not apply or is unknown. It shall be displayed as blank, never as the text "NULL". 60 customers have no region and 22 have no fax. *[Expected]*

REQ-SEM-05 The employee with a NULL `reports_to` is the top of the organization chart. Reports that list managers shall show "None" for this employee. *[Expected]*

REQ-SEM-06 A product shall be shown as "Out of stock" when `units_in_stock` is 0 and as "Reorder" when `units_in_stock` is below `reorder_level`. One active product is out of stock. *[Expected]*

REQ-SEM-07 Money amounts shall be displayed rounded to 2 decimal places. No currency is stored; all amounts are in one implied currency. *[Expected]*

REQ-SEM-08 Delivery address fields on an order (`ship_name`, `ship_address`, `ship_city`, `ship_country`) describe where the order was sent and may differ from the customer address. *[Expected]*

9. Known Gaps

The following rules are part of the intended business behavior but are not enforced by the schema. Tests that write invalid data will be accepted by the database, so a test expecting an error for these cases will fail by design.

- Discount range, quantity sign and the 0 or 1 value of `discontinued` (Section 6.4).
- Date order between `order_date`, `required_date` and `shipped_date` (REQ-ORD-05).
- Orders with no lines (REQ-ORD-06) and upper case customer identifiers (REQ-CUS-03).
- No identity columns: callers must choose new key values themselves (REQ-INT-02).
- Money stored as single precision `real`, which can produce small rounding differences (REQ-SQL-21).

Appendix A. Requirement Summary

Area	Enforced	Expected	Identifiers
System	2	1	REQ-SYS-01 to 03
Products	4	3	REQ-PRD-01 to 07
Suppliers	2	0	REQ-SUP-01 to 02
Customers	5	1	REQ-CUS-01 to 06
Employees	5	1	REQ-EMP-01 to 06
Orders	2	5	REQ-ORD-01 to 07

Area	Enforced	Expected	Identifiers
Order details	4	3	REQ-ODT-01 to 07
Shippers and reference	3	0	REQ-SHP-01, REQ-REF-01 to 02
SQL behaviors	23	2	REQ-SQL-01 to 25
Integrity	3	0	REQ-INT-01 to 03
Boundaries	6	5	REQ-BND-01 to 11
Configuration	1	4	REQ-CFG-01 to 05
Semantics	0	8	REQ-SEM-01 to 08